Kubernetes HPA with HTTP Metrics: End-to-End Scrape & Scale

This reference document maps out the target metrics source (the application exposing metrics), the scraper deployment (Prometheus monitoring configuration), and the destination consumer (Horizontal Pod Autoscaler).

1. Source: Application Scraping Manifests

The target application must expose Prometheus metrics at an HTTP endpoint (usually **/metrics**) and be discoverable via a Kubernetes Service with proper scraping annotations.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: metrics-app
        image: nginx:alpine # Replace with your HTTP-app exposing /metrics
        ports:
        - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: web-app-svc
  namespace: default
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "8080"
    prometheus.io/path: "/metrics"
spec:
  ports:
  - port: 8080
    targetPort: 8080
  selector:
    app: web-app
```

2. Scraper Configuration: Prometheus Rule

Prometheus discovers the service endpoints using the annotations defined above, aggregates the raw HTTP counter data, and computes a rate. The Prometheus Adapter then exposes this to the Custom Metrics API.

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: web-app-metrics-rules
  namespace: default
spec:
  groups:
  - name: web-app.rules
    rules:
    - record: http_requests_per_second
      expr: sum(rate(http_requests_total[1m])) by (pod)
```

3. Destination: Horizontal Pod Autoscaler (HPA)

The HPA controller continuously watches the Custom Metrics API endpoint for **http_requests_per_second** and automatically evaluates the desired replica count, using stabilization windows to avoid sudden thrashing.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-http-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 15
  metrics:
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second
      target:
        type: AverageValue
        averageValue: "50"  # Target: 50 HTTP requests per second per pod
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300  # Cool-down for 5 minutes before scaling down
      policies:
      - type: Percent
        value: 100
        periodSeconds: 15
```